课程笔记：智能体的长上下文建模

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：75:39
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=17
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：为什么长上下文是智能体的核心问题 **2**
1.1 本章小结 2

2 上下文为什么会爆炸：从线性历史到二次计算 **2**
2.1 本章小结 3

3 推理基础：prefill、decode 与服务体系 **3**
3.1 本章小结 3

4 能装下不等于能用好：有效上下文与 agent 失效 **3**
4.1 本章小结 4

5 长上下文架构：把二次计算压下来 **4**
5.1 滑动窗口注意力 4
5.2 线性注意力与 DeltaNet 4
5.3 稀疏注意力 5
5.4 KV 压缩：GQA、MQA 与 MLA 5
5.5 本章小结 5

6 训练长上下文：数据比架构更难 **5**
6.1 位置编码：绝对位置、RoPE 与 NoPE 6
6.2 数据配方与上下文并行 6
6.3 本章小结 7

7 推理侧优化：前缀缓存是长上下文的生命线 **7**
7.1 KV cache 与前缀命中 7
7.2 两个代表系统：PagedAttention 与 RadixAttention 7
7.3 给 harness 设计者的三条红线 7
7.4 本章小结 8

8 上下文压缩（Compaction）：装不下时的最后一招 **8**
8.1 什么该保留 8
8.2 策略与风险 8
8.3 评测陷阱：一个真实事故 8
8.4 本章小结 9

9 总结与延伸 **9**

1 引言：为什么长上下文是智能体的核心问题

上一讲把工具接进了模型，工具会不断往对话里塞入观察结果：读文件的结果、搜索的结果、命令的输出。于是上下文像滚雪球一样变长，而语言模型的注意力计算随长度**二次增长**。这一讲的主题就是：当 agent 的上下文长到几十万甚至上百万 token 时，怎么办？

主讲人把长上下文问题拆成两个不能混为一谈的挑战：

1. **容量（capacity）**：模型的架构、位置编码与训练数据能不能让它真正「装下并利用」这么多证据；这需要架构设计、训练策略与评测三方面配合。
2. **效率（efficiency）**：即使模型能装下，服务系统是否付得起这个成本——注意力的二次复杂度、KV cache 的显存、推理的时延与费用。

本讲路线图

上下文为什么爆炸（增长模型 + 真实会话数据）→ 推理基础与服务体系（prefill/decode、TTFT/TPOT、KV cache）→「能装下」不等于「能用好」（needle-in-a-haystack 与 agent 失效案例）→ 长上下文架构（混合模型、滑动窗口、线性注意力、稀疏注意力、KV 压缩）→ 长上下文训练（数据、位置编码、上下文并行）→ 推理侧优化（前缀缓存、PagedAttention、RadixAttention、缓存感知路由）→ 上下文压缩（compaction）与其评测陷阱。

1.1 本章小结

长上下文是 agent 的「容量与效率」双重问题：容量决定模型能否用好证据，效率决定系统能否承担成本。本讲按「为什么长、怎么装、怎么省、装不下怎么办」四层展开。

2 上下文为什么会爆炸：从线性历史到二次计算

每一次 agent 调用都要把「系统提示 + 全部历史 + 新观察」重新作为输入送进模型。历史是线性增长的，但每一步都要对全部历史做注意力，于是总 token 访问量是二次的。课堂给了一个极简算例：每次调用新增 5 个 token、初始系统提示 1 个 token，则输入长度依次是 6、11、16、21、26，五步下来累计处理 **80K** token——这还只是五次调用。

真实 agent 会话更夸张。主讲人统计了自己用 coding agent（OpenHands）的 **1,500 个会话**（都是解决单个任务的短会话，不是你连续用几天的长会话），平均每个会话消耗 **77,922 个 token**。这 7.8 万 token 的构成非常说明问题：

组成部分	占比
系统提示 + 工具描述	23%（约 1.5–1.8 万 token，视工具数量在 0.8–2 万之间）
工具结果（读文件、命令输出等）	37%
工具调用（写代码、读代码等）	20%
用户消息	9%
推理轨迹（模型思考）	9%
模型直接回复用户	2%

两个观察值得记住：第一，**系统提示与工具描述占了近四分之一**，一个「有主见」的 coding agent 光提示词就是一两万 token；第二，**模型花在「想」上的 token 约是花在「说」上的五倍**（思考 9%

对回复 2%），而真正的大头是被工具结果和工具调用吃掉的（57%）。这解释了为什么 agent 的成本与上下文管理强相关。

2.1 本章小结

上下文爆炸的根源是「每步重算全部历史」：线性历史带来二次计算。真实 coding agent 单任务会话平均已接近 8 万 token，其中工具结果与调用占一多半、系统提示占近四分之一——这两个数字直接决定了后面所有优化的方向。

3 推理基础：prefill、decode 与服务体系

在谈长上下文之前，先把推理过程拆清楚。语言模型推理分为两个阶段：

- **prefill（预填充）**：一次性处理输入的所有 token，为每个位置计算 key/value 对；这一步是并行的，决定**首 token 时延**。
- **decode（解码）**：逐 token 生成输出，每生成一个 token 就把它的 key/value 追加到缓存，下一步只用新 token 的 query 去和全部历史 KV 做注意力；这一步是串行的，决定**每 token 时延**。

服务系统用四个指标衡量好坏：**TTFT**（time to first token，首 token 时延）、**TPOT**（time per output token，每输出 token 时延）、**吞吐**（单位时间完成的 token/请求数）、**成本**。课堂强调这些指标之间存在权衡，而且要按场景取舍：交互式 coding CLI 里你非常在意 TTFT/TPOT；后台跑一整晚的 agent 则完全不在乎时延，只在乎吞吐与成本。

不同推理供应商的差异也很大：同一个模型（课堂展示的是 GLM 5.2 的 OpenRouter 页面），有的供应商每秒几十 token，有的只有 3 token/秒，也有做到每秒几百 token 的。对 agent 来说这直接决定「用起来爽不爽」。

3.1 本章小结

推理分 prefill 与 decode 两阶段，分别对应 TTFT 与 TPOT；吞吐与成本是系统级指标。长上下文优化本质上是同时压这四项：架构与缓存降低单步成本，调度与并行提升吞吐。

4 能装下不等于能用好：有效上下文与 agent 失效

很多模型宣称支持 128K 甚至 1M 上下文，但「宣称的支持长度」与「有效利用长度」是两件事。课堂先复习了标准注意力：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right)V,$$

其中 X 是输入表示，W_Q, W_K, W_V 是投影矩阵，d 是每个头的维度，M 是因果掩码（未来位置为 $-\infty$）。注意力要让每个 token 与所有历史 token 两两比较，因此对序列长度 n 是 $O(n^2)$；当 $n = 10^6$ 时，光比较次数就是 10^{12} 量级，再乘上向量乘法，计算量可达 10^{20} 量级。

那模型在长上下文上表现如何？课堂用一个经典实验说明：把 Paul Graham 的大量文章拼成长上下文，中间插入一句「旧金山最好吃的三明治在哪」，然后问模型同一个问题。上下文越长，弱模

型答得越差——这类评测被称为 **needle in a haystack**（大海捞针）。更系统的基准是 **RULER** 与 **HELMET** 等。

这个问题在 agent 场景里格外致命，因为用户是在对话**中途**不断追加指令的。课堂现场收集的失败案例很典型：忘记会话开头给出的初始指令；用户在中途说「不要做某件事」，模型转头就做了；最严重的一例是用户明确说「不要删这个」，模型还是删掉了。原因就是它已经「忘了」那条约束。

4.1 本章小结

「支持 1M 上下文」不等于「能用好 1M」；needle-in-haystack、RULER、HELMET 等评测反复显示模型在超长上下文下会退化。agent 场景中用户指令分散在会话中段，遗忘的代价可能是删库，所以长上下文管理是刚需而不是优化项。

5 长上下文架构：把二次计算压下来

几乎所有长上下文架构都在做同一件事：**给每个时间步的比较次数加上一个上限**，把 $O(n^2)$ 压成 $O(W \cdot n)$，其中 W 是「窗口」或「比较次数上限」。按上限发生的位置，模型分为两类：

- **局部（local）模型**：计算被限制在局部窗口内，感受野靠层数逐层扩展；
- **全局（global）模型**：存在贯穿整个序列的全局计算（稀疏或稠密）。

今天几乎所有主流模型都是**混合体**：大量局部计算 + 周期性的一次全局计算，再重复。课堂给的代表性模型包括：Qwen3.5（Gated DeltaNet 局部 + 稀疏检索全局）、GLM 5.3（delta net + 稀疏注意力）、Kimi K3（delta net + 稠密注意力）、Nemotron（Mamba + 稠密注意力）、Inkling（滑动窗口 + 稠密注意力）、DeepSeek（窗口注意力 + 稀疏分支交错）；局部与全局的计算比大致在 3:1 到 5:1 之间。

5.1 滑动窗口注意力

概念上最简单的做法：把序列切成块（例如每块 512 token），当前块只关注自己与固定数量的前序块。单个注意力层没有变化，但**层与层叠加后感受野逐步扩大**——第 1 层能看到前一两块，第 2 层的前序块本身又包含了更早的信息，于是窗口随深度增长。这也是滑动窗口注意力被广泛使用的原因。

5.2 线性注意力与 DeltaNet

线性注意力的思路是：把标准注意力里的 softmax 去掉，只保留 query 与 key 的点积，于是可以利用矩阵乘法的结合律，把所有历史 key-value 的贡献压缩成一个**状态矩阵**：

$$S_t = S_{t-1} + k_t v_t^\top, \qquad o_t = q_t^\top S_t.$$

这样每来一个新 token 只需用常数时间更新状态，复杂度从 $O(n^2)$ 降到 $O(n)$，本质上变成了一个循环神经网络（RNN）。代价是表达能力下降：没有 softmax 的相对比较，模型不再能「挑出」最相关的历史 token，只能把历史压进一个固定大小的状态里。

DeltaNet 是对这个状态更新的改进。它把状态矩阵看作「query 到 value 的映射」，并希望这个映射能预测出当前的 value：用当前 key 问状态矩阵，得到预测值 $\hat{v}_t$；用真实值 v_t 减去预测得到误差；再用这个误差去修正状态：

$$\hat{v}_t = S_{t-1}^{\top} k_t, \qquad S_t = S_{t-1} + \beta_t \, k_t \, (v_t - \hat{v}_t)^{\top},$$

其中 β_t 类似学习率，控制单步修正的幅度。直观理解：把状态矩阵朝着「让这个 key 输出正确 value」的方向推一步，但不要推太狠。

线性注意力还有一个隐患：很久以前的异常 key 可能会在未来产生很大影响，且无法自动淡化。解决办法是加**衰减（decay）**：每一步把状态矩阵乘上一个衰减系数 α_t，越久远的信息权重越低。Gate 化的 delta net 让衰减是学出来的；GLM、Kimi 等模型更进一步，用**逐元素（per-element / channel-wise）**的衰减——不同通道可以有不同的遗忘速度，于是有的信息能保留很久、有的很快被清掉。

5.3 稀疏注意力

稀疏注意力不关注全部历史，只关注其中一部分。两条路线：

1. **固定模式**：按固定步长稀疏（例如每隔 512 token 取一个位置）。整体仍是二次的，但系数小 512 倍，工程上够用；这是最早的一批稀疏注意力工作（OpenAI 多年前的做法）。
2. **内容相关（content-dependent）**：先用很便宜的方式算一遍「粗注意力」——例如把 key/query 下投影到很低的维度，在低维上算注意力、选出 top-K 相关位置，再对 top-K 做完整注意力。DeepSeek 用的是这一类；它便宜、能写成一个 kernel，但需要两遍计算。

5.4 KV 压缩：GQA、MQA 与 MLA

除了改注意力结构，还能从 KV cache 本身下手。标准多头注意力有 H 个头，每个头各有一份 K 和 V，显存开销是 $2 \times H \times d$ 每 token。三种压缩方式：

- **GQA（Grouped Query Attention）**：让多个 query 头共享一组 KV 头，KV 头数从 H 降到 $H/4$ 或 $H/8$；这是目前最主流、性价比最高的做法；
- **MQA（Multi-Query Attention）**：极端情况，所有头共享一份 KV（GQA 的特例）；
- **MLA（Multi-head Latent Attention）**：不减少头数，而是把每个头的 K/V 维度下投影压小；可以与 GQA 叠加使用。

5.5 本章小结

长上下文架构的统一范式是混合：用局部计算（滑动窗口、线性注意力/DeltaNet、Mamba）承担大部分计算，用周期性全局计算（稀疏或稠密注意力）保留远距离信息通路；同时用 GQA/MQA/MLA 压缩 KV cache。代价是表达能力与工程复杂度之间的权衡。

6 训练长上下文：数据比架构更难

架构只是「能装」，真正让模型「会用」要靠训练。而训练长上下文的第一难题是**数据**：互联网上真正连贯的 128K 长度文本并不多——一本 128K token 的书已经超过大多数维基百科条目，而网上的书并没有那么多。

标准做法是「由短到长」：先在较短序列上做预训练，再用更长的文档做继续训练，最后做长上下文适配（long adaptation）。这里有个容易踩的坑：**packed sequences（把多篇文档拼进一个序**

列）对吞吐有用，但对学「长上下文连贯性」没用——模型并不需要、也不应该学会跨文档注意，适配阶段真正需要的是天然连贯的长文本。

6.1 位置编码：绝对位置、RoPE 与 NoPE

位置编码是长上下文训练的第二个难点。

- **绝对位置编码**：给每个位置一个位置向量（正弦/余弦等函数生成），每个 token 的表示都带上自己的绝对位置。
- **RoPE（旋转位置编码）**：用旋转操作把「相对距离」编码进 query/key 内积，使注意力只依赖相对位置。数学优雅、被大量模型采用；具体做法是通过旋转矩阵与复数运算实现，这里不展开。
- **NoPE（完全不用位置编码）**：一个反直觉但被证实的结论。经典说法是「Transformer 必须有位置编码，否则位置 1 和位置 5 的同一个词表示相同」；但在**自回归 + 因果掩码**的 Transformer 里，这个说法不再成立——位置 2 的「cat」只看到前 1 个 token，位置 5 的「cat」看到前 4 个 token，第一层注意力输出的表示就已经不同，顺序信息由因果掩码隐式携带。带衰减的线性注意力（gated delta net）进一步强化了这种顺序差异。需要注意的是：**非自回归**的 Transformer（所有 token 互相注意）仍然需要位置编码。

如果模型用了位置编码，就要做**长度扩展**。RoPE 的初衷是只依赖相对位置，但把训练长度 L 扩展到 sL 时有两个问题：其一，模型从没见过相距超过 L 的两个 token 互相注意，远距离注意力会失准；其二，相对位置编码只是一部分，因果掩码等机制同样参与，所以不能指望 RoPE 自动外推。常见修法有：

1. **位置插值**：RoPE 里有一个控制每步旋转角度的参数 θ，把它除以扩展倍数 s（即 $\theta' = \theta/s$），相当于把位置「拉伸」后压缩回原范围；
2. **YaRN** 等更精细的方案：不把所有频率一起插值，而是按波长分组——长波长维度多插值、短波长维度保持本地分辨率，兼顾远距离与局部精度。

6.2 数据配方与上下文并行

公开模型披露的长上下文数据配方大致包括：清洗后的长文档、代码库（代码天然连贯且 token 量大）、同主题多文档语料、专利（公开且可链接）、**agent 轨迹**（agent 交互本身就会产生很长的轨迹）、以及合成任务。训练通常采用 8K → 64K → 256K → 1M 的尺度递进策略。至于**数据混合比例与训练策略**，闭源厂商完全不披露，开源厂商也只公布架构、不公布配方——这是「能跑模型」与「能训出好模型」之间的信息差。

另外，训练超长序列需要**上下文并行（context parallelism）**：把长序列切到不同设备上，每台设备算一段的 KV 再传给下一台。除了它，常见的并行还有数据并行、张量并行、流水线并行、专家并行。上下文并行在 32K 以上会变得重要，也会带来一个很现实的矛盾：**建模团队想要 Kimi Delta Attention 这类新结构，基础设施团队却要为它实现可反向传播的上下文并行 kernel**——主讲人自己就遇到过「想用小尺寸 Qwen 跑上下文并行，但网上根本没有对应 delta net kernel」的窘境。标准注意力、滑动窗口注意力的 kernel 容易实现，非标准操作才是难点。

6.3 本章小结

训练长上下文难在数据而不是架构：真正连贯的长文本稀缺，packed 序列救不了连贯性；位置编码上要理解 RoPE 的相对性、NoPE 在自回归 Transformer 中可行这一反直觉结论，以及位置插值/YaRN 这类长度扩展手段；数据配方是各家最保密的部分；工程上超过 32K 后需要上下文并行，但新架构的 kernel 支持常常滞后。

7 推理侧优化：前缀缓存是长上下文的生命线

7.1 KV cache 与前缀命中

agent 的每一次调用都是「上一轮的全部输入 + 输出 + 新的工具结果」。如果每次都重算全部历史，代价巨大；而历史中绝大部分前缀是不变的，因此可以把已经算好的 KV 缓存起来复用。课堂给出的量化直觉是：到第 50 步时，若不做特殊处理，历史部分是新增部分的 **50 倍**；缓存让计算从二次变成接近线性。

价格上，主流厂商的**缓存 token 价格约为未缓存 token 的十分之一**（课堂引用的官方定价表口径）。这也意味着缓存命中率是 agent 成本的生死线：课堂讨论给的经验值是 **90%–95% 的命中率算健康**，低于这个值可能就有地方在漏钱。另外，输出 token 的价格通常是输入的数倍，而「缓存输入」与「输出」之间可以差到 50 倍量级。

7.2 两个代表系统：PagedAttention 与 RadixAttention

- **vLLM 的 PagedAttention**：借鉴操作系统虚拟内存分页，把 KV cache 切成固定大小的物理块，用块表（block table）把「逻辑上连续的序列」映射到「物理上离散的块」。好处是可以并行服务大量序列、生成结束时可整块回收，显著降低显存碎片。
- **SGLang 的 RadixAttention**：用基数树（radix tree）管理前缀，自动复用已缓存的前缀（例如多轮对话、共享系统提示的请求）。它最初为聊天场景设计，如今对 agent 更重要——agent 会话更长、更频繁。

多机部署时还要做**缓存感知路由**：请求进来后，路由到「持有最匹配 KV cache」的机器，再由该 worker 处理。缓存命中率还取决于缓存淘汰策略——厂商不会永久保存你的缓存，常见 TTL 是几分钟到一小时（课堂提到 Anthropic 早期约 5 分钟、现在可选到 1 小时但更贵）；而**定价并不随命中率变化**，所以「被过早淘汰」的成本由用户承担。

7.3 给 harness 设计者的三条红线

如果只是在写 agent 而不是部署模型，缓存同样决定了你的成本。前缀缓存生效的前提是**整个前缀逐 token 完全相同**，所以：

1. **不要每步改 system message**：例如把当前时间/日期拼进系统提示——它每步都变，等于让缓存完全失效。需要当前时间就拼到最后一条用户消息或工具结果里。
2. **不要每步换模型**：这一步用强模型、后几步用便宜模型看似省钱，但下次再用强模型时缓存已经失效，仍要付全价；模型切换要非常克制。
3. **可以动态改工具列表**，但要知道这会改变前缀、影响命中。

7.4 本章小结

前缀 KV 缓存是长上下文 agent 的成本生命线：缓存 token 约为未缓存价格的 1/10，健康命中率 90%–95%；系统侧用 PagedAttention 管碎片、用 RadixAttention 复用前缀、用缓存感知路由跨机命中；harness 侧最重要的一条纪律是「保持前缀稳定」。

8 上下文压缩（Compaction）：装不下时的最后一招

当上下文超过模型或预算能承受的长度时，最后一招是**压缩**：把历史「压」成更短的表示，让 agent 带着压缩后的状态继续工作。目标可以形式化为：压缩后的状态要足够小（有界），同时让后续动作序列与压缩前「相同、相似或更好」。

8.1 什么该保留

课堂给出一张很实用的分区图：

- **锚点（anchors）**：会话最开始用户说的话（初始目标、约束），通常必须保留——很多 agent 遗忘开头指令，其实是 harness 的设计缺陷；
- **检查点（checkpoint）**：把中段历史总结成结构化摘要；
- **最近窗口（recent tail）**：最近若干步的动作与观察，通常原样保留；
- **证据库（evidence store）**：完整历史通常落盘保存，压缩只影响上下文、不销毁记录；需要时可以让 agent 回去读原始历史。

8.2 策略与风险

触发时机有三种：token 阈值（如超过 200K 就压缩）、供应商硬上限（被迫）、手动触发（coding agent 的 `/compact` 命令）。摘要形式有自由文本、结构化摘要（要求必须包含某些字段），也有厂商把压缩结果加密成不可读的形式（防止别人窃取压缩算法）。

风险在于**反复压缩会持续丢信息**：第一次压缩可能无损，压缩多次后 agent 会逐渐「跑偏」。这也解释了为什么用户看到「compacting context」提示往往不开心。

8.3 评测陷阱：一个真实事故

主讲人分享了 OpenHands 早期做压缩（当时叫 compression）的教训：研究团队花约一个月做出一个压缩算法，在 **SWE-bench** 上评测「分数完全不掉、还省了大量 token」，看起来非常成功。但上线到真实系统后出现大量低级错误：

- 每次压缩后都**重复发 pull request**——因为压缩把「已经发过 PR」这件事忘了，同一个功能开了五个 PR；
- 忘记用户**在会话中途**给出的指令——因为评测集是单轮任务、只有一条用户消息，压缩时把多条用户消息都压掉了。

结论是：**只在基准上评测不够**。压缩算法必须用能覆盖真实使用形态的鲁棒基准来评，并且要在真实环境里跑一段时间；「benchmark 不掉分」不等于「产品里不出错」。

8.4 本章小结

压缩是长上下文的兜底手段：保留锚点、压缩中段、保留最近窗口、把完整历史放进证据库；触发方式有阈值/硬上限/手动三种；反复压缩会累积信息损失；评测必须覆盖多轮真实交互，否则会把「单轮基准上的成功」错当成「产品可用」。

9 总结与延伸

把全讲收束成一张系统栈（课堂最后一页幻灯片的分层）：

1. **增长（growth）**：agent 循环不断重复指令、动作与环境观察，上下文自然爆炸；
2. **推理（inference）**：prefill、decode、注意力和 KV cache 各有不同瓶颈（TTFT、TPOT、显存、带宽）；
3. **架构（architecture）**：局部、循环（线性注意力/DeltaNet/Mamba）、稀疏、压缩等机制决定「哪些历史在物理上可被访问」；
4. **缓存（caching）**：缓存复用稳定前缀、加速 prefill，但不扩大模型能看到的窗口；
5. **压缩（compaction）**：在不删除可恢复历史的前提下压缩当前上下文，用摘要 + 最近窗口 + 证据库续命。

把本讲压缩成五句话

1. agent 上下文是二次爆炸的：线性历史 + 每步全量注意力；真实 coding 会话平均已近 8 万 token。
2. 长上下文有两个独立问题：容量（架构/位置编码/训练数据）与效率（服务成本）。
3. 主流架构是「局部 + 周期全局」的混合模型；线性注意力把 $O(n^2)$ 变成 $O(n)$，代价是表达能力，DeltaNet 用预测误差 + 衰减弥补。
4. 训练侧难在连贯长文本稀缺与位置编码外推；推理侧靠前缀 KV 缓存（约 1/10 价格、90%+ 命中率）与 Paged/Radix 管理。
5. 压缩是兜底，但评测必须在多轮真实形态上做——否则会重演「benchmark 不掉分、线上重复开 PR」的事故。

延伸思考：(1) 对做 agent 训练的人，长上下文直接影响 rollout 的设计——轨迹越长，KV 显存、截断策略、组内奖励归因都会变形；(2) 对做 harness 的人，本讲给出的「保持前缀稳定」「别频繁换模型」「压缩要保留锚点」三条纪律是成本与正确性的最低要求；(3) 对做研究的人，长上下文架构（线性注意力、稀疏、混合）与长上下文训练（数据配方、位置编码）之间仍有大量未解问题。下一讲将把视角从「单次会话的上下文」拉长到「跨会话的记忆与技能」。